

[Algorithm]

자료구조와 알고리즘

인하대학교 소프트웨어융합공학과

김종훈 교수



인하대학교
INHA UNIVERSITY

이 장에서 다룰 내용

1

알고리즘의 필요성

2

추상 데이터 타입

3

자료구조

4

알고리즘

5

성능 분석 및 알고리즘 예제

□ 알고리즘의 필요성

❖ 컴퓨터 분야에서 알고리즘을 왜 배워야 하는가?

- 얼마나 정확하고 효율적으로 소프트웨어의 개발과 사용 및 관리가 이루어지는가?
 - 컴퓨터가 효율적으로 문제를 처리하기 위해서는 문제를 정의하고 분석하여 그에 대한 최적의 프로그램을 작성해야 한다.
 - 자료구조와 알고리즘에 대한 개념과 활용 능력 필요!

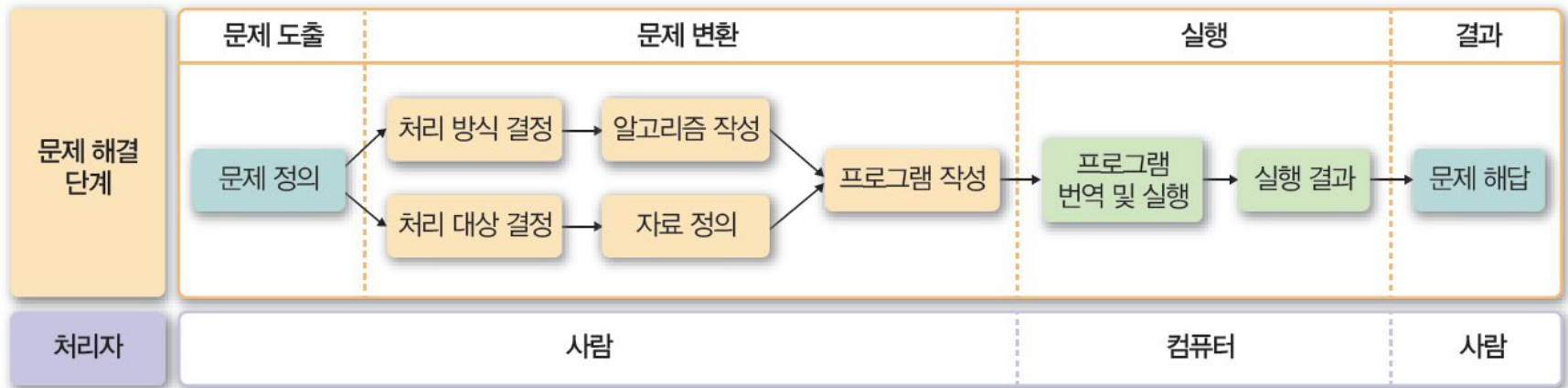


그림 1-2 문제 해결 과정

□ 추상 자료형

❖ 뇌의 추상화 기능

- 기억할 대상의 구별되는 특징만을 단순화하여 기억하는 기능
 - 예: 뇌의 추상화 기능



❖ 컴퓨터를 이용한 문제해결에서의 추상화

- 크고 복잡한 문제를 단순화시켜 쉽게 해결하기 위한 방법
- 자료 추상화(Data Abstraction)
 - 처리할 자료, 연산, 자료형에 대한 추상화 표현
 - 자료 : 프로그램의 처리 대상이 되는 모든 것을 의미
 - 연산: 어떤 일을 처리하는 과정. 연산자에 의해 수행
 - 예) 더하기 연산은 + 연산자에 의해 수행
 - 자료형 : 처리할 자료의 집합과 자료에 대해 수행할 연산자의 집합
 - 정수 자료형의 자료 : 정수의 집합. {..., -1, 0, 1, ...}
 - 연산자 : 정수에 대한 연산자 집합. {+, -, x, ÷, mod}

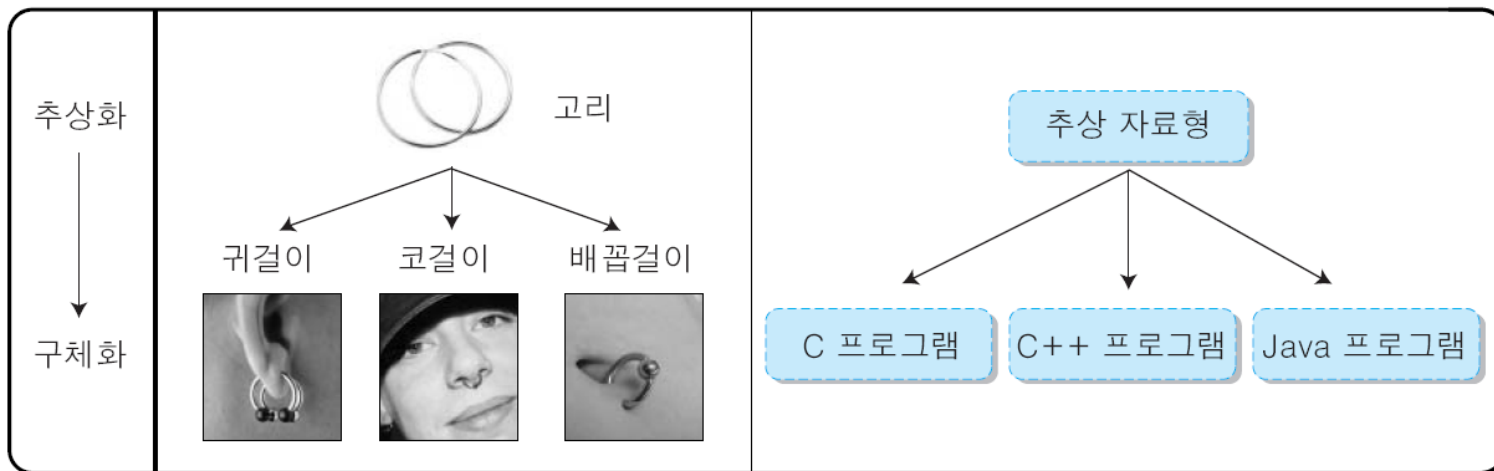
□ 추상 자료형

❖ 추상 자료형(ADT, Abstract Data Type)

- 자료와 연산자의 특성을 논리적으로 추상화하여 정의한 자료형

❖ 추상화와 구체화

- 추상화 – “무엇(what)인가?”를 논리적으로 정의
- 구체화 – “어떻게(how) 할 것인가?”를 실제적으로 표현





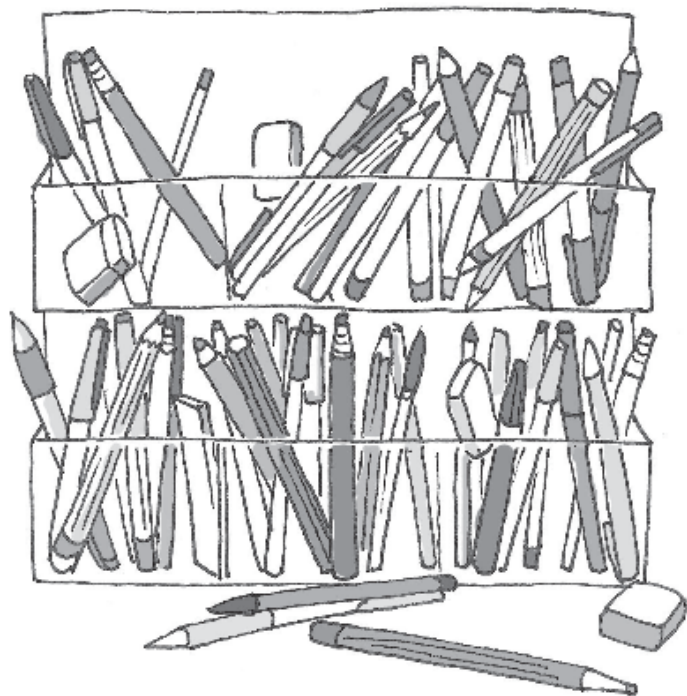
[그림 2-7] 추상화의 구체화의 예 ②

■ 자료와 연산에 있어서의 추상화와 구체화의 관계

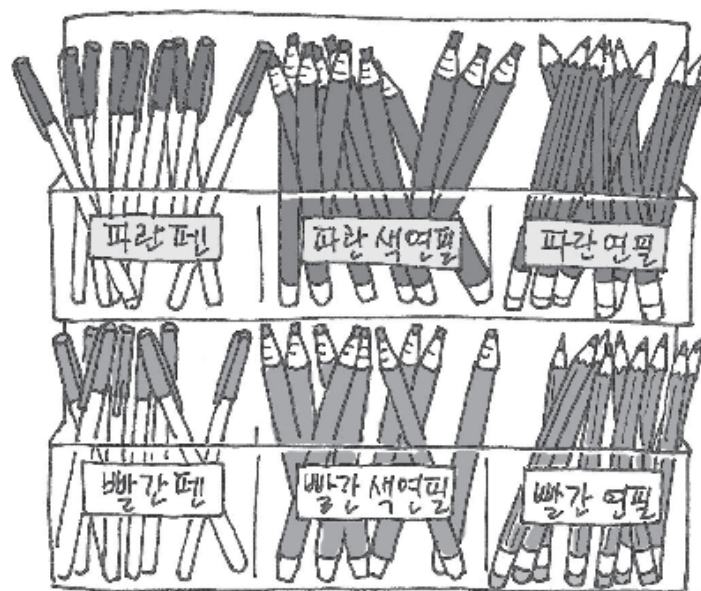
	자료	연산
추상화	추상 자료형	알고리즘 정의
구체화	자료형	프로그램 구현

❖ 자료구조의 개념

- 자료를 효율적으로 표현하고 저장하고 처리할 수 있도록 정리하는 것



(a) 자료구조를 적용하기 전

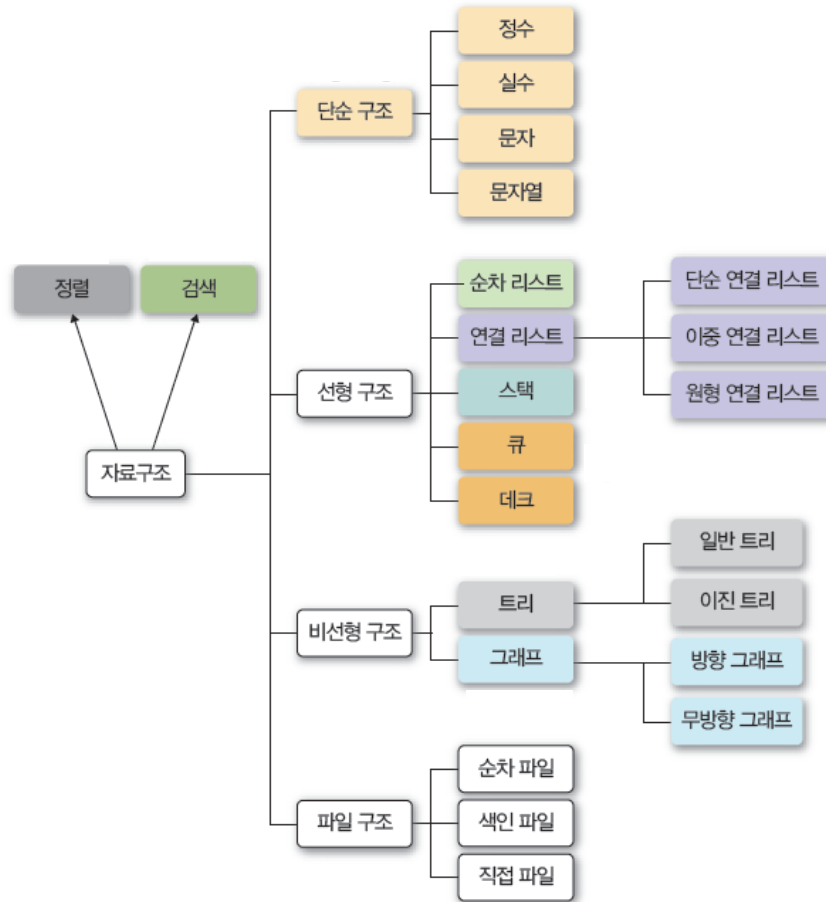


(b) 자료구조를 적용한 후

생활 속에서 자료구조를 적용한 예

❖ 자료구조의 분류

- 자료의 형태에 따른 분류



❖ 알고리즘

- 문제 해결 방법을 추상화하여 각 절차를 논리적으로 기술해 놓은 명세서
- 알고리즘이란 특정한 일을 수행하는 명령어들의 유한 집합을 의미함
- 알고리즘 중 가장 효율적인 알고리즘을 찾는 것이 중요함
- 수학에서는 문제를 풀기 위해 정의나 정리들을 활용하는 데 비해 컴퓨터에서는 수행 가능한 효율적인 알고리즘을 사용함

■ 알고리즘의 시작

- 알고리즘의 개념은 1830년 배비지(Babbage)와 러브레이스(Lovelace)에 의해 처음으로 제기됨
- 1973년 컴퓨터 과학자인 크누스(Knuth)가 그의 저서 《컴퓨터 프로그래밍의 기술(The Art of Computer Programming)》에서 체계적으로 정립함



정의 12-1

알고리즘(algorithm)이란 주어진 문제를 해결하기 위해 필요한 여러 가지 단계들을 체계적으로 명시해 놓은 것을 말한다. 알고리즘의 사전적 의미는 '어떤 문제를 해결하는 한 방법의 상세한 특징을 기술하는 것'이다.

■ 알고리즘의 조건

- 입력(input):
 - 알고리즘 수행에 필요한 자료가 외부에서 입력으로 제공될 수 있어야 한다.
- 출력(output)
 - 알고리즘 수행 후 하나 이상의 결과를 출력해야 한다.
- 명확성(definiteness)
 - 수행할 작업의 내용과 순서를 나타내는 알고리즘의 명령어들은 명확하게 명세되어야 한다.
- 유한성(finiteness)
 - 알고리즘은 수행 뒤에 반드시 종료되어야 한다.
- 효과성(effectiveness)
 - 알고리즘의 모든 명령어들은 기본적이며 실행이 가능해야 한다.

❖ 알고리즘의 표현 방법

- 자연어를 이용한 서술적 표현 방법
- 순서도(Flow chart)를 이용한 도식화 표현 방법
- 프로그래밍 언어를 이용한 구체화 방법
- 가상코드(Pseudo-code)를 이용한 추상화 방법
- 누구나 이해할 수 있도록 명확하게 기술하는 것이 매우 중요함



여기서 잠깐!!

알고리즘의 응용 예

- 지하철 노선의 최단 경로를 찾는 법
- 전자레인지 등 전자 제품의 사용 설명서
- 단계적 음식 조리법
- 덧셈, 곱셈 등의 수학적 계산
- 수많은 일상생활에서 만나는 문제들의 효과적인 해결 방안 등



절차

[요리 재료]

자료

스펀지케이크(20×20cm) 1개, 크림치즈 200g, 달걀 푼 물 2개 분량, 설탕 3큰술, 레몬즙·바닐라에센스 1큰술씩, 딸기시럽(딸기 500g, 설탕 1½ 컵, 레몬즙 1작은술), 딸기 1개, 플레인 요구르트 2큰술

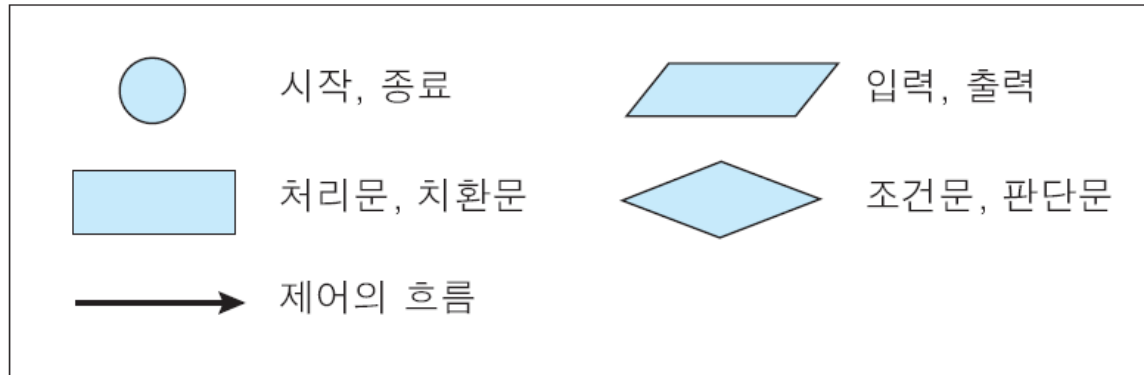
[요리법]

알고리즘

- ① 케이크 틀의 가장자리에 필름을 돌린 다음 스펀지케이크를 놓는다.
- ② 볼에 크림치즈를 넣고 거품기로 젓다가 달걀 푼 물과 설탕 3큰술을 세번에 나누어 넣으면서 크림 상태로 만든다.
- ③ ②에 레몬즙과 바닐라에센스를 넣고 살짝 저은 다음 ①에 붓는다. 이것을 180℃의 오븐에 넣고 20분 정도 굽는다.
- ④ 냄비에 슬라이스한 딸기와 설탕 1½ 컵을 넣고 끓이다가 약한 불에서 끓어붙지 않도록 저으면서 거품을 걷어낸다. 되직해지면 레몬즙을 넣고 차게 식힌다.
- ⑤ 접시에 치즈케이크를 한 조각 담고 ④의 시럽을 뿌린 다음 플레인 요구르트와 딸기를 얹어낸다 .

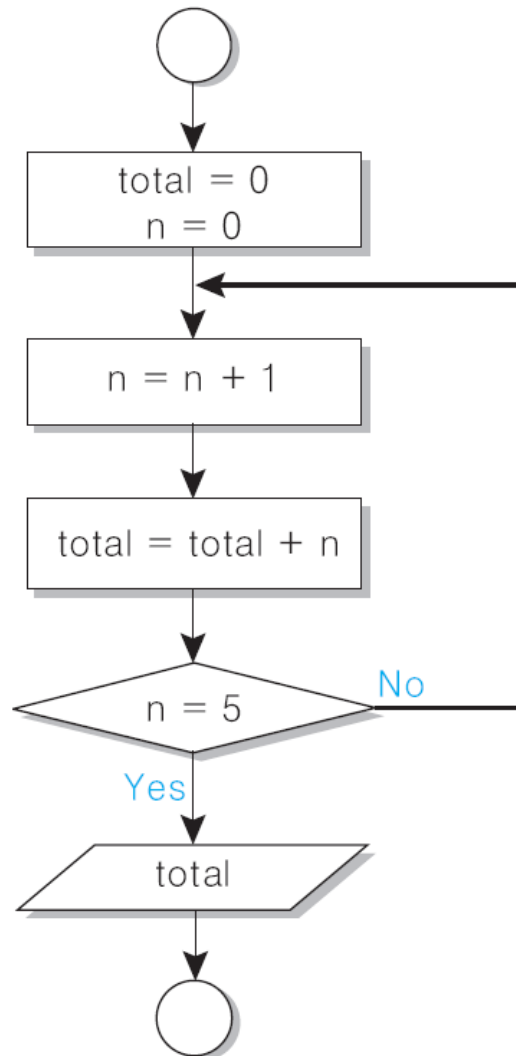
❖ 순서도를 이용한 알고리즘의 표현

■ 순서도에서 사용하는 기호



- 장점 : 알고리즘의 흐름 파악이 용이함
- 단점 : 복잡한 알고리즘의 표현이 어려움

- 순서도 예) 1부터 5까지의 합을 구하는 알고리즘



❖ 가상코드를 이용한 알고리즘의 표현

- 가상코드, 즉 알고리즘 기술언어(ADL)를 사용하여 프로그래밍 언어의 일반적인 형태와 유사하게 알고리즘을 표현
- 특정 프로그래밍 언어가 아니므로 직접 실행은 불가능
- 일반적인 프로그래밍 언어의 형태이므로 원하는 특정 프로그래밍 언어로의 변환 용이

❖ 가상코드의 형식

■ 기본 요소

• 기호

- 변수, 자료형 이름, 프로그램 이름, 레코드 필드 명, 문장의 레이블 등을 나타냄
- 문자나 숫자의 조합. 첫 문자는 반드시 영문자 사용

• 자료형

- 정수형과 실수형의 수치 자료형, 문자형, 논리형, 포인터, 문자열 등의 모든 자료형 사용

• 연산자

- 산술연산자, 관계연산자, 논리연산자

■ 대입문

- 사용형식

변수 ← 값 ;

- 대입연산자(←)의 오른쪽에 있는 값(식의 계산 결과값이나 변수값)을 대입연산자(←)의 왼쪽에 있는 변수에 저장

■ 조건문

- 조건에 따라 실행할 명령문이 결정되는 선택적 제어 구조

- if 문

- 형식

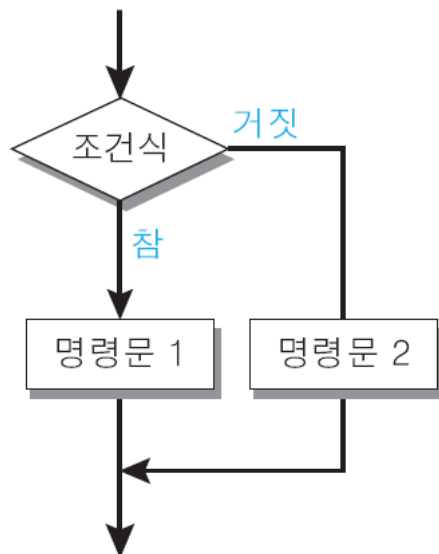
if (조건식) then 명령문 1;
else 명령문 2;

또는

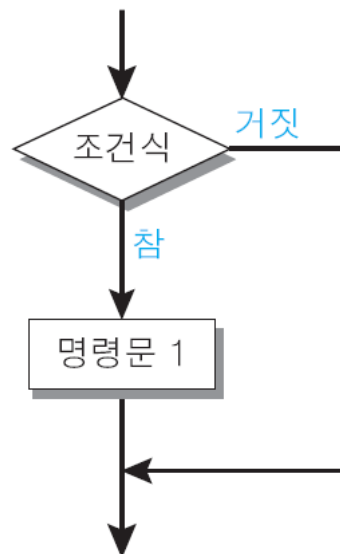
if (조건식) then 명령문 1;

if (조건식 1) then 명령문 1;
else if (조건식 2) then 명령문 2;
else 명령문 3;

• if 문의 제어 흐름



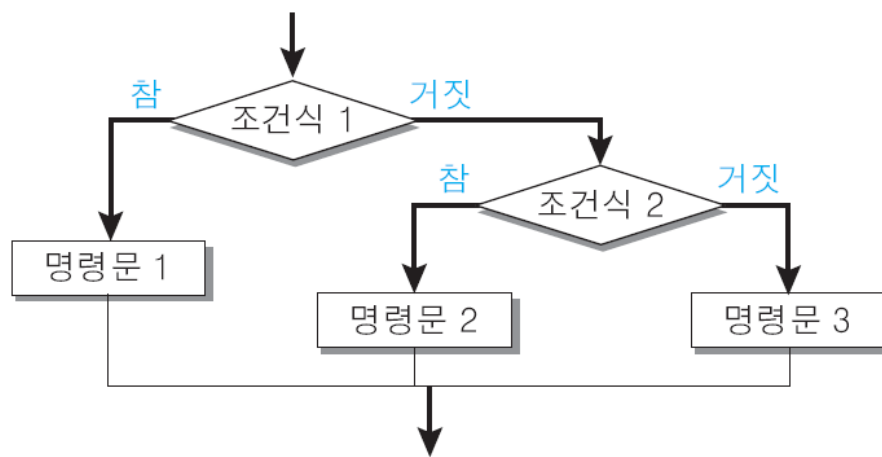
또는



➤ 중첩 if 문

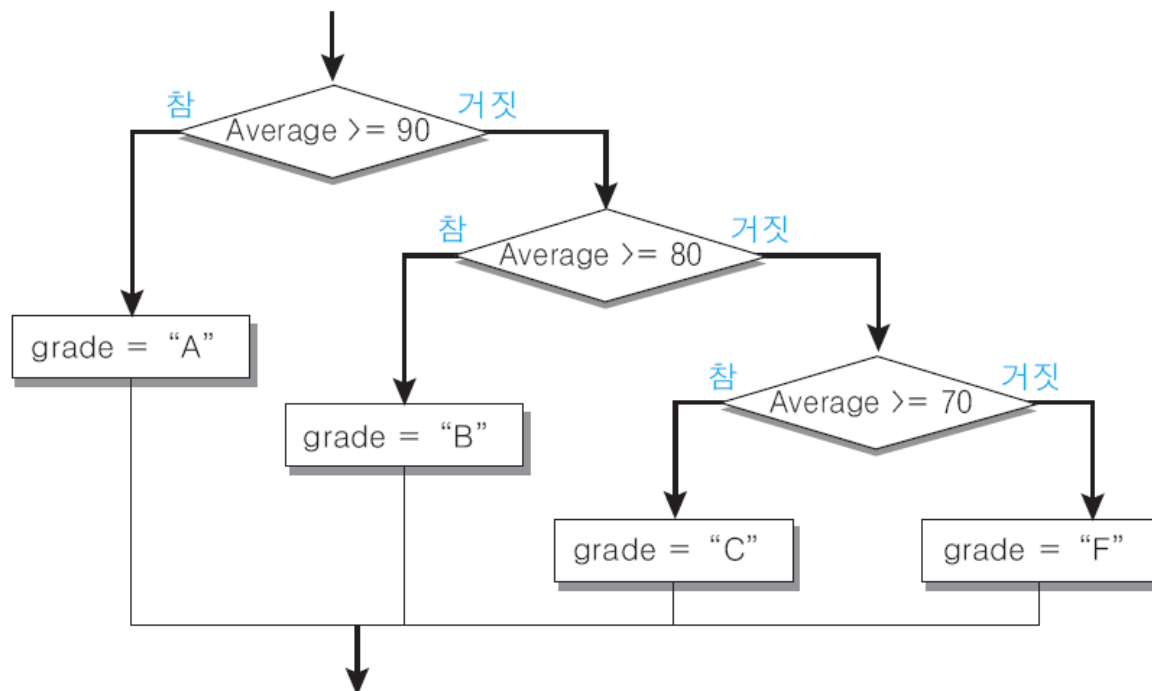
```

if (조건식 1) then 명령문 1 ;
else if (조건식 2) then 명령문 2;
else 명령문 3;
  
```



➤ if문 사용 예) 평균 점수에 따른 등급 계산하기

```
if Average >= 90 then grade = "A" ;  
else if Average >= 80 then grade = "B" ;  
    else if Average >= 70 then grade = "C" ;  
        else grade = "F" ;
```

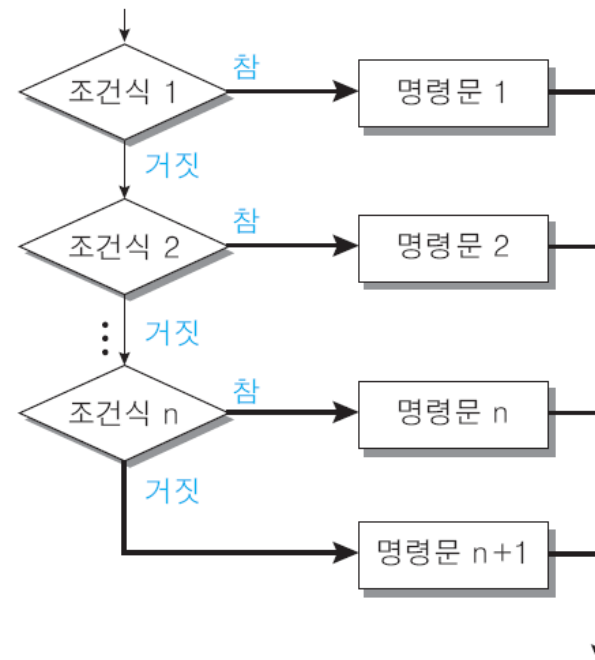


- case 문

- 여러 조건식 중에서 해당 조건을 찾아서 그에 대한 명령문을 수행
- 중첩 if 문으로 표현 가능
- 형식

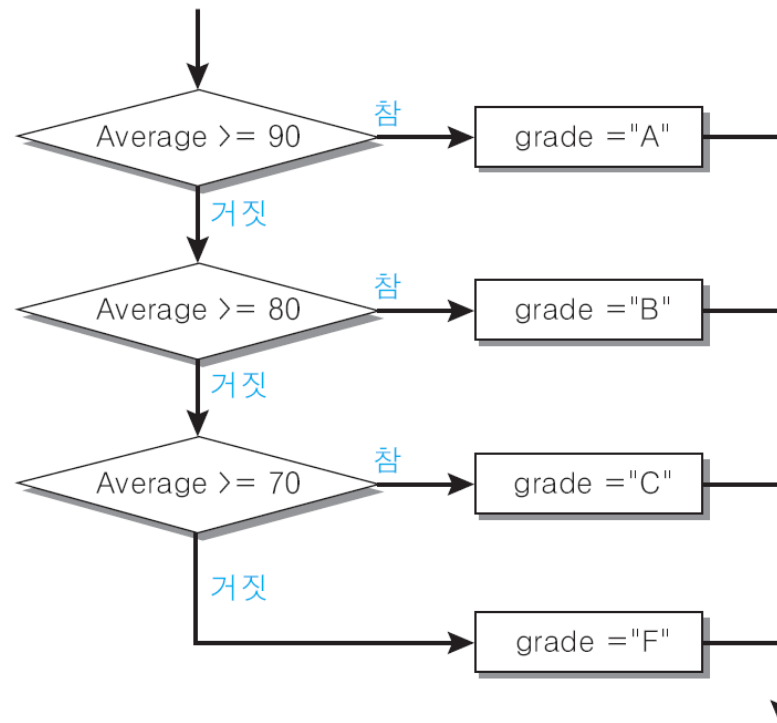
```
case {  
    조건식 1 : 명령문 1;  
    조건식 2 : 명령문 2;  
    ...  
    조건식 n : 명령문 n;  
    else    : 명령문 n+1;  
}
```

- case 문의 제어 흐름



➤ case 문 사용 예) 평균 점수에 따른 등급 계산하기

```
case {  
    Average >= 90 : grade = "A"  
    Average >= 80 : grade = "B"  
    Average >= 70 : grade = "C"  
    else          : grade = "F"  
}
```



■ 반복문

- 일정한 명령을 반복 수행하는 루프(loop) 형태의 제어구조

- for 문

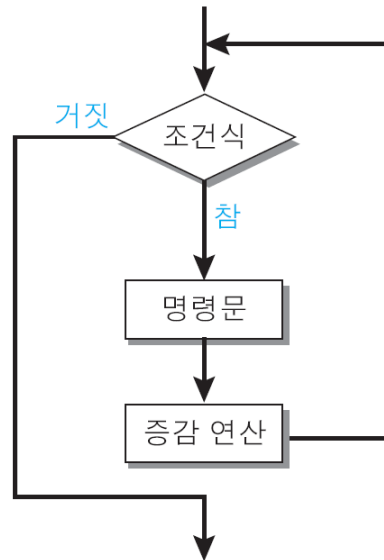
- 형식 : `for (초기값 ; 조건식 ; 증감값) do 명령문 ;`

- 초기값 : 반복문을 시작하는 값

- 조건식 : 반복 수행 여부를 검사하는 조건식

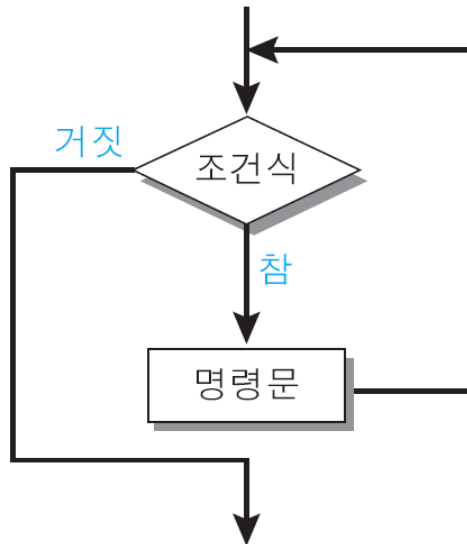
- 증감값 : 반복 회수를 계산하기 위해서 반복문을 한번 수행할 때마다 증가 또는 감소시키는 값

- for 문의 제어 흐름



- while 문

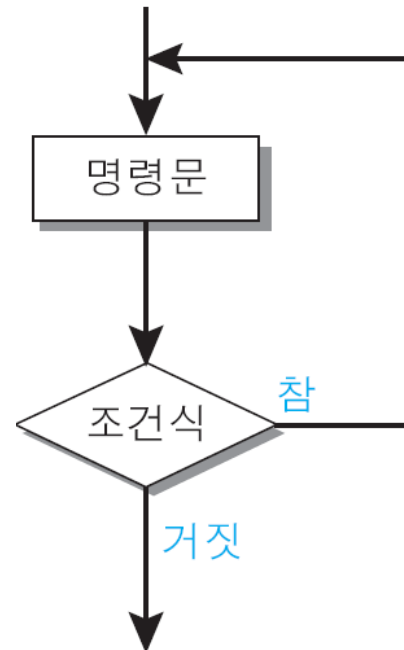
- 형식 : `while (조건식) do 명령문 ;`
- 조건식이 참인 동안 명령문을 반복 수행
- while 문의 제어 흐름



- do-while 문

- 형식 :

```
do 명령문 ;  
while (조건식) ;
```
- 일단 명령문을 한번 실행한 후에 조건식을 검사하여 조건식이 참인 동안 명령문을 반복 수행
- do-while 문의 제어 흐름



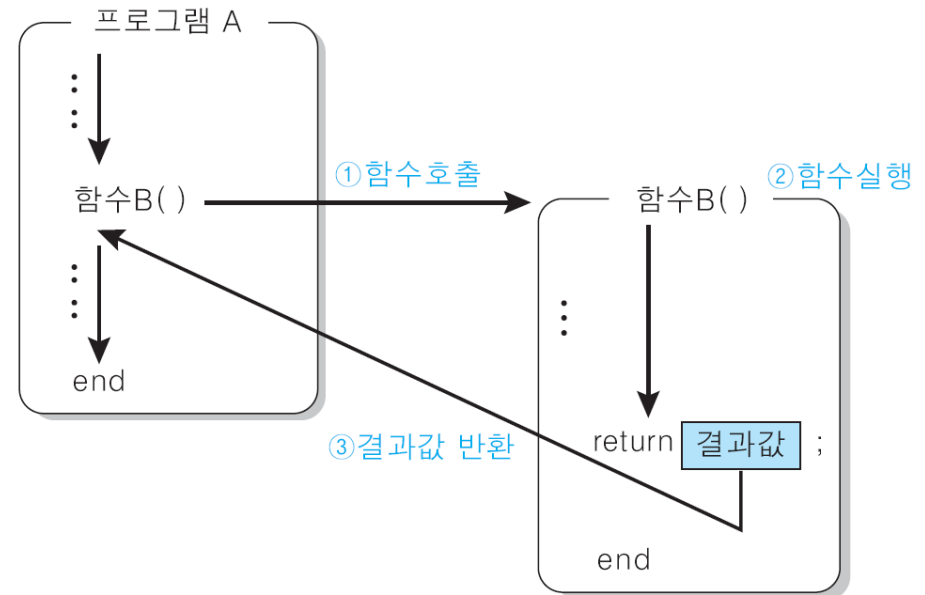
■ 함수문

- 처리작업 별로 모듈화하여 만든 부프로그램

- 형식 :

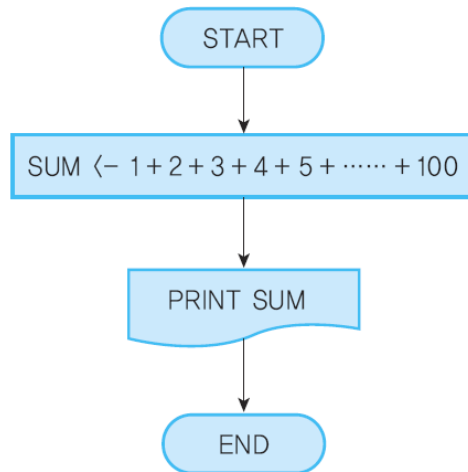
```
함수이름 (매개변수)
    명령문 ;
    ...
    return 결과값 ;
end
```

- 함수의 호출과 실행 및 결과값 반환에 대한 제어 흐름

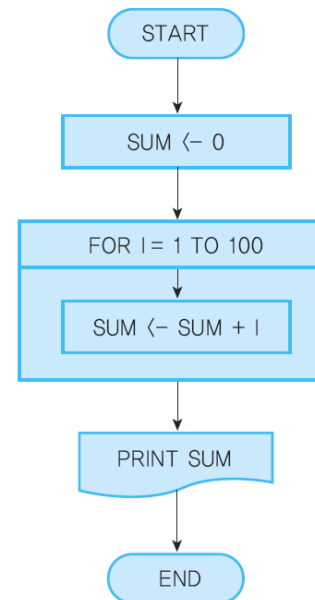


□ 알고리즘의 효율성

- 어떤 문제가 주어졌을 때 그 문제를 풀기 위한 방법론인 알고리즘이 항상 하나만 있는 것은 아님
- 컴퓨터 프로그램의 경우에는 어떤 알고리즘이 가장 효율적인지를 선택되어야 함
- 일반적으로 수행 시간, 메모리 용량, 자료의 종류, 프로그래머의 성향에 따라 가장 알맞은 알고리즘이 선택됨



〈그림 12.1〉 일반적인 계산 방법



〈그림 12.2〉 for 문을 사용한 계산 방법

□ 알고리즘의 효율성

- 알고리즘에 대한 평가와 비교는 컴퓨터 프로그램을 통한 문제 해결에 있어서 매우 중요한 관심 분야임
- 어떤 문제를 해결하는 데 있어서 두 가지 질문을 해봄으로 알고리즘을 분석할 수 있음
 - ① 어떤 문제의 해결에 있어서 주어진 알고리즘을 사용하는 데 드는 비용이 얼마인가?
 - ② 그 문제를 해결하는 데 비용이 가장 적게 드는 알고리즘은 무엇인가?
- 비용이란 연산하는 데 필요한 **시간**과 **기억 장소**의 크기를 말함
- 주어진 문제를 해결하는 방법에는 여러 가지 알고리즘이 있으나 그 중에서 비용이 적게 드는 알고리즘을 찾기 위해서는 **효율성 분석 (performance analysis)**이 필요함

□ 알고리즘의 효율성

- 효율성을 분석하기 위해서는 알고리즘 수행 시 필요한 시간 복잡도 (time complexity)와 공간 복잡도(space complexity)의 두 가지 요소를 검토함
- 수행 시간과 그에 따르는 기억 장소의 크기는 알고리즘이 처리하는 입출력 자료의 크기에 따라 달라짐
- 일반적으로 알고리즘을 분석할 때 입력의 개수를 n 으로 생각하고 효율성을 그에 대한 함수로 나타냄



여기서 잠깐!!

최근에는 컴퓨터 하드웨어의 발달로 알고리즘 분석에서 어떤 문제를 해결하는 데 필요한 기억 장소의 크기에는 그리 큰 비중을 두고 있지 않다. 따라서 여기서는 주로 알고리즘의 수행 시간에 대하여 분석하고 살펴보기로 한다.

❖ 알고리즘 성능 분석 방법

■ 공간 복잡도

- 알고리즘을 프로그램으로 실행하여 완료하기까지 필요한 총 저장 공간량
- 공간 복잡도 = 고정 공간 + 가변 공간

■ 시간 복잡도

- 알고리즘을 프로그램으로 실행하여 완료하기까지의 총 소요시간
- 시간 복잡도 = 컴파일 시간 + 실행 시간
 - 컴파일 시간 : 프로그램마다 거의 고정적인 시간 소요
 - 실행 시간 : 컴퓨터의 성능에 따라 달라질 수 있으므로 실제 실행시간보다 명령문의 실행 빈도수에 따라 계산
- 실행 빈도수의 계산
 - 지정문, 조건문, 반복문 내의 제어문과 반환문은 실행시간 차이가 거의 없으므로 하나의 단위시간으로 갖는 기본 명령문으로 취급



예제 12-1

다음과 같이 n 개의 양의 정수 중에서 가장 작은 수를 찾는 알고리즘에서 기억 장소의 크기와 수행 횟수를 구해보자.

```
Find_MIN (int array[ ], int MIN)
{
    int i ;
    MIN = array[0];
    for (i = 1; i < n; i++)
        if (MIN > array[i]) MIN = array[i];
    return (MIN);
}
```



예제 12-2

다음의 각각의 프로그램들에서 `sum += i;` 명령문에 대한 수행 횟수를 구해보자.

(1) `sum += i;`

(2) `for (i = 0; i < n; i++)`

`sum += i;`

(3) `for (i = 0; i < n; i++)`

`for (j = 0; j < n; j++)`

`sum += i;`



■ 피보나치 수열 알고리즘의 빈도수 구하기

fibonacci(n)

[알고리즘 2-1]

```
01  if (n<0) then
02      stop ;
03  if (n≤1) then
04      return n ;
05  f1 ← 0 ;
06  f2 ← 1 ;
07  for (i←2 ; i≤n ; i←i+1) do {
08      fn←f1+f2 ;
09      f1←f2 ;
10      f2←fn ;
11  }
12  return fn ;
13 end
```

- $n < 0$, $n = 0$, $n = 1$ 의 경우에 대한 실행 빈도수
 - for 반복문이 수행되지 않기 때문에 실행 빈도수가 작다.

행 번호	$n < 0$	$n = 0$	$n = 1$
1	1	1	1
2	1	0	0
3	0	1	1
4	0	1	1
5~13	0	0	0

- $n > 1$ 의 일반적인 경우에 대한 실행 빈도수

➤ n 에 따라 for 반복문 수행

행 번호	실행 빈도수	행 번호	실행 빈도수
1	1	8	$n-1$
2	0	9	$n-1$
3	1	10	$n-1$
4	0	11	0
5	1	12	1
6	1	13	0
7	n		

➤ 총 실행 빈도수

$$= 1 + 0 + 1 + 0 + 1 + 1 + n + (n-1) + (n-1) + (n-1) + 0 + 1 + 0 = 4n + 2$$



정의 12-2

알고리즘의 복잡성을 측정하는 데 있어서 일반적으로 빅오(Big-Oh) 표현을 사용하며, 대문자 O 를 써서 표시한다.

예를 들어, $f(n) = O(n)$ 로 표시가 되었다면, 우리는 이것을 $f(n)$ 의 차수(order)가 n 이라 하며, 'Big-Oh of n ' 이라고 읽는다.

■ 시간 복잡도 표기법

- 빅-오(Big-Oh) 표기법 사용
- 빅-오(Big-Oh) 표기법 순서
 - ① 실행 빈도수를 구하여 실행시간 함수 찾기
 - ② 실행시간 함수의 값에 가장 큰 영향을 주는 n 에 대한 항을 선택하여
 - ③ 계수는 생략하고 O (Big-Oh)의 오른쪽 괄호 안에 표시
- 피보나치 수열의 시간 복잡도 = $O(n)$
 - ① 실행시간 함수 : $4n+2$
 - ② n 에 대한 항을 선택 : $4n$
 - ③ 계수 4는 생략하고 O (Big-Oh)의 오른쪽 괄호 안에 표시 : $O(n)$



예제 12-4

다음 함수들을 O 의 개념으로 나타내어보자.

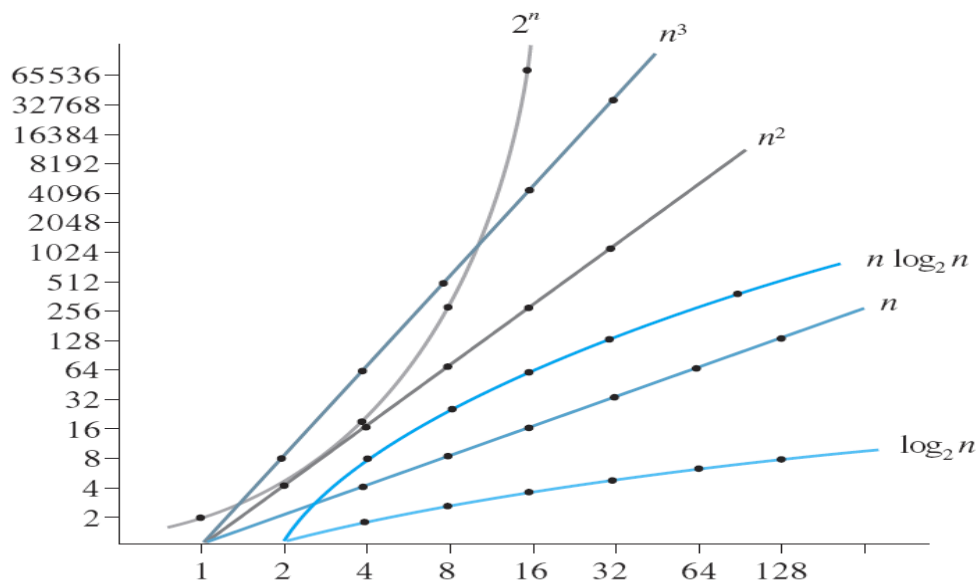
(1) $3n + 2$

(2) $n^2 + 16n + 1$

(3) $16n + 5n\log_2 n$

■ 각 실행 시간 함수에서 n값의 변화에 따른 실행 빈도수 비교

$\log n$	<	n	<	$n \log n$	<	n^2	<	n^3	<	2^n
0		1		0		1		1		2
1		2		2		4		8		4
2		4		8		16		64		16
3		8		24		64		512		256
4		16		64		256		4096		65536
5		32		160		1024		32768		4294967296



〈그림 12.3〉 입력 크기에 따른 각 함수의 증가 비율

■ 입력크기에 따른 알고리즘 수행시간 비교 예

표 1-10 시간 복잡도에 따른 알고리즘 수행 시간 비교 예

입력 크기 n	알고리즘 수행 시간				
	n	nlogn	n^2	n^3	2^n
10	10^{-8} 초	3×10^{-8} 초	10^{-7} 초	10^{-6} 초	10^{-6} 초
30	3×10^{-8} 초	2×10^{-7} 초	9×10^{-7} 초	3×10^{-5} 초	1초
50	5×10^{-8} 초	3×10^{-7} 초	3×10^{-6} 초	10^{-4} 초	13일
100	10^{-7} 초	7×10^{-7} 초	10^{-5} 초	10^{-3} 초	4×10^{13} 년
1,000	10^{-6} 초	10^{-5} 초	10^{-3} 초	1초	3×10^{283} 년
10,000	10^{-5} 초	10^{-4} 초	10^{-1} 초	17분	
100,000	10^{-4} 초	2×10^{-3} 초	10초	12일	
1,000,000	10^{-3} 초	2×10^{-2} 초	17분	32년	

- 알고리즘을 분석하는 데 있어서 가장 중요한 것은 주어진 문제를 해결할 수 있는 알고리즘 중 각 알고리즘의 수행 시간을 계산하여 가장 적은 수행 시간이 걸리는 알고리즘을 찾는 것임



Thank You
